Going through this program has pushed me to think about software development in a way I didn't when I first started. Early on I was focused on just getting code to work but over time I started caring more about how it was organized, why certain approaches are better than others, and what it actually means to write something that another person could read and build on. That shift in thinking is probably the most valuable thing I'm taking away from this experience and the ePortfolio gave me a place to show that growth in a concrete way.

Collaboration is something I've been building toward throughout the program. Most of my project work has been individual but working through code reviews, getting feedback on my designs, and learning to document my decisions clearly so others can follow my thinking has given me a foundation to build on. Communicating with stakeholders is something the written reflections and video code review helped develop. Having to explain technical decisions to an audience that might not be deep in the code forces you to think about what matters and how to say it clearly. Data structures and algorithms came through directly in the Hangman project where I made deliberate choices like replacing a vector with an unordered set for faster lookup and using strings instead of raw arrays to simplify memory management. Software engineering principles showed up in the way I refactored the project into separate classes each with a single responsibility. The database work demonstrated practical skills in persistent data storage and application integration. Security showed up through defensive programming habits like validating user input, handling file and database errors gracefully, and anticipating edge cases rather than assuming everything would work perfectly.

The artifacts in this portfolio all come from the same project, a C++ Hangman game I originally built as a final project for CIS 125 at Berkshire Community College. I chose to keep enhancing the same program across all three artifacts because I wanted to show how improvements build on each other over time rather than presenting three unrelated pieces of work. The first enhancement focused on software engineering and design. Refactoring a single overloaded class into four separate classes each responsible for one thing. The second focused on data structures and algorithms. Replacing a vector with an unordered set for constant time duplicate detection and swapping raw character arrays for strings to eliminate manual memory management. The third integrated a SQLite database to replace the text file the program used to pull words from adding difficulty levels and demonstrating skills in database design and persistent storage. Together these three enhancements tell a story of a program that grew from a functional but basic piece of code into something organized, efficient, and connected to real industry tools and practices.